

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 6 Transacciones y Control de Concurrency

Alumno: Tomás Rodeghiero
Año: 2026

Resolución basada en el Teórico 8 (Transacciones y Control de Concurrency) y en experimentación sobre MySQL y PostgreSQL.

Índice

1. Introducción	2
2. Ejercicio 1: Planificaciones concurrentes y serializabilidad	2
2.1. Marco teórico aplicado	2
2.2. Planificación 1: serializable por conflicto	3
2.3. Planificación 2: no serializable por conflicto	3
3. Ejercicio 2: Protocolo de bloqueo de dos fases puro	4
3.1. Marco teórico aplicado	4
3.2. Planificación 2PL puro propuesta	5
3.3. Verificación de las fases por transacción	5
4. Ejercicio 3: Protocolo de marcas temporales	5
4.1. Marco teórico aplicado	6
4.2. Asignación inicial	6
4.3. Schedule propuesto y evolución	6
4.4. Resultado final	7
5. Ejercicio 4: UPDATE y ROLLBACK en MySQL	7
5.1. Marco teórico aplicado	7
5.2. Ejecución sugerida	7
5.3. Resultado esperado	8
5.4. Verificación opcional desde una segunda sesión	8
6. Ejercicio 5: Dos sesiones MySQL en SERIALIZABLE	8
6.1. Marco teórico aplicado	8
6.2. Prueba propuesta (<code>cliente con nro_cliente = 1</code>)	8
7. Ejercicio 6: Lectura no repetible	9
7.1. Marco teórico aplicado	9
7.2. Prueba (<code>producto.cod_producto = 102</code>)	10
7.3. Niveles de aislamiento que lo permiten	10
8. Ejercicio 7: Deferrable Constraints en PostgreSQL	11
8.1. Marco teórico aplicado	11
8.2. a) Comportamiento con esquema normal	11
8.3. b) Esquema que evita la excepción	12
8.4. c) Esquema que fuerza excepción	12
8.5. Consulta de control	12
9. Ejercicio 8: MVCC en PostgreSQL (<code>xmin / xmax</code>)	13
9.1. Marco teórico aplicado	13
9.2. Prueba (<code>vehiculo.patente = 'AA123BB'</code>)	13
9.3. Interpretación de <code>xmin/xmax</code>	14
9.4. Restaurar el dato (opcional)	14
10. Conclusiones	14

1. Introducción

El Práctico 6 cierra el cuatrimestre con los conceptos sobre los que se apoya cualquier base de datos multiusuario: **transacciones** y **control de concurrencia**. Los ocho ejercicios se reparten en dos mitades complementarias:

- **Ejercicios 1–3:** parte teórica. Planificaciones concurrentes, serializabilidad por conflicto, protocolo de dos fases y protocolo de marcas temporales.
- **Ejercicios 4–8:** parte práctica. Pruebas reales en MySQL (transacciones, niveles de aislamiento, lectura no repetible) y en PostgreSQL (constraints deferibles, MVCC con $xmin/xmax$).

El soporte teórico es el Teórico 8: definición de transacción, propiedades ACID, estados de una transacción, schedules, conflictos, grafos de precedencia, recuperabilidad, protocolos de bloqueo (2PL y variantes) y protocolo de marcas temporales.

Notación. A lo largo del documento se usa la notación corta $r_i(Q)$ para “ T_i lee Q ” y $w_i(Q)$ para “ T_i escribe Q ”. Las asignaciones locales (por ejemplo $X := X - N$) no generan conflictos entre transacciones y se omiten en el análisis de schedules. Los bloqueos se denotan $LX_i(Q)$ (exclusivo) y $LC_i(Q)$ (compartido), y $UX_i(Q)$ / $UC_i(Q)$ para sus desbloques.

Verificación de las resoluciones previas

Antes de redactar este documento se revisaron los ocho README. Conclusiones de la revisión:

- **Ejercicio 1:** las dos planificaciones propuestas y sus grafos de precedencia están correctos. Sin cambios de fondo; se completaron diagramas de grafos.
- **Ejercicio 2:** la planificación con 2PL puro respeta la fase de crecimiento y la de decrecimiento de cada transacción. Sin errores.
- **Ejercicio 3:** tabla de evolución de $MT-E$ y $MT-L$ correcta paso a paso; no hay rollbacks en el schedule propuesto. Sin errores.
- **Ejercicios 4–8:** las pruebas SQL son consistentes con el comportamiento real de MySQL/InnoDB y PostgreSQL. Se ajustó la redacción para enmarcar cada experimento dentro del concepto teórico que ilustra (atomicidad, bloqueos, niveles de aislamiento, MVCC, constraints diferidas).

2. Ejercicio 1: Planificaciones concurrentes y serializabilidad

Enunciado. Dadas:

- T_1 : leer(X); $X:=X-N$; escribir(X); leer(Y); $Y:=Y+N$; escribir(Y);
- T_2 : leer(X); $X:=X+M$; escribir(X);

dar dos planificaciones concurrentes, una serializable por conflicto y otra no serializable, justificando con el grafo de precedencia.

2.1. Marco teórico aplicado

Dos instrucciones $I_i \in T_i$ e $I_j \in T_j$ están en **conflicto** si acceden al mismo ítem Q y *al menos una* es una escritura. Quedan en conflicto los pares $r-w$, $w-r$ y $w-w$ sobre Q ; $r-r$ no genera conflicto.

Una planificación es **serializable por conflicto** si es equivalente, mediante intercambios de operaciones no conflictivas, a alguna planificación serial. El *test del grafo de precedencia* construye

un nodo por transacción y agrega un arco $T_i \rightarrow T_j$ cada vez que una operación de T_i precede a una de T_j con la que está en conflicto. La planificación es serializable por conflicto si y sólo si el grafo es acíclico.

2.2. Planificación 1: serializable por conflicto

$$S_1 = r_1(X), w_1(X), r_2(X), w_2(X), r_1(Y), w_1(Y)$$

Forma extendida (con asignaciones locales):

Paso	Transacción	Operación
1	T_1	leer(X)
2	T_1	$X := X - N$
3	T_1	escribir(X)
4	T_2	leer(X)
5	T_2	$X := X + M$
6	T_2	escribir(X)
7	T_1	leer(Y)
8	T_1	$Y := Y + N$
9	T_1	escribir(Y)

Análisis de conflictos. Sólo hay accesos compartidos sobre X (a Y accede únicamente T_1). Conflictos sobre X :

- $w_1(X)$ antes de $r_2(X) \Rightarrow T_1 \rightarrow T_2$.
- $w_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.
- $r_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.

Grafo de precedencia.



El grafo no tiene ciclos: S_1 es **serializable por conflicto**. Es equivalente a la planificación serial T_1 seguida de T_2 .

2.3. Planificación 2: no serializable por conflicto

$$S_2 = r_1(X), r_2(X), w_1(X), w_2(X), r_1(Y), w_1(Y)$$

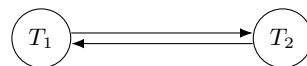
Forma extendida:

Paso	Transacción	Operación
1	T_1	leer(X)
2	T_2	leer(X)
3	T_1	$X := X - N$
4	T_1	escribir(X)
5	T_2	$X := X + M$
6	T_2	escribir(X)
7	T_1	leer(Y)
8	T_1	$Y := Y + N$
9	T_1	escribir(Y)

Análisis de conflictos.

- $r_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.
- $r_2(X)$ antes de $w_1(X) \Rightarrow T_2 \rightarrow T_1$.
- $w_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.

Grafo de precedencia.



Hay un ciclo $T_1 \leftrightarrow T_2$: S_2 **no es serializable por conflicto**. La intuición detrás del problema es que cada transacción “ve” el X original (no el ya modificado por la otra), y al final ambas sobrescriben el ítem con un valor que no refleja el efecto de la otra.

3. Ejercicio 2: Protocolo de bloqueo de dos fases puro

Enunciado. Las transacciones son las del Ejercicio 1: dar una planificación concurrente que respete el protocolo de **dos fases puro** (2PL básico).

3.1. Marco teórico aplicado

El **Protocolo de Bloqueo de Dos Fases** (2PL) divide la vida de toda transacción en dos etapas:

- **Fase de crecimiento:** la transacción puede solicitar bloqueos, pero no puede liberarlos.
- **Fase de decrecimiento:** la transacción puede liberar bloqueos, pero no puede solicitar nuevos.

El protocolo garantiza planificaciones *serializables en cuanto a conflicto*. Como ambas transacciones escriben sobre X , hace falta bloqueo **exclusivo** sobre X para las dos; T_1 además bloquea Y en exclusivo porque también lo escribe.

3.2. Planificación 2PL puro propuesta

Paso	Transacción	Operación
1	T_1	$LX_1(X)$
2	T_1	$LX_1(Y)$
3	T_1	$\text{leer}(X)$
4	T_1	$X := X - N$
5	T_1	$\text{escribir}(X)$
6	T_2	$LX_2(X)$ ($\dashrightarrow$ espera)
7	T_1	$UX_1(X)$ (inicia decrecimiento de T_1)
8	T_2	$LX_2(X)$ (concedido)
9	T_2	$\text{leer}(X)$
10	T_1	$\text{leer}(Y)$
11	T_2	$X := X + M$
12	T_1	$Y := Y + N$
13	T_2	$\text{escribir}(X)$
14	T_2	$UX_2(X)$
15	T_2	commit
16	T_1	$\text{escribir}(Y)$
17	T_1	$UX_1(Y)$
18	T_1	commit

3.3. Verificación de las fases por transacción

T_1 .

- Crecimiento: pasos 1 y 2 (adquiere X y Y).
- Punto de viraje: paso 7 (libera X).
- Decrecimiento: del paso 7 en adelante; T_1 no vuelve a pedir bloqueos, sólo opera con los que ya tenía y al final libera Y (paso 17).

T_2 .

- Crecimiento: paso 8 (consigue X cuando T_1 lo libera).
- Decrecimiento: paso 14 (libera X).
- No vuelve a pedir bloqueos luego de liberar.

Conclusión. Ambas transacciones tienen un único punto donde pasan de crecer a decrecer: la planificación es 2PL puro. La ejecución es además concurrente (los pasos 8–14 entrelazan operaciones de T_2 con T_1) y el orden efectivo equivalente es serial $T_1 \rightarrow T_2$ sobre X .

2PL puro no exige que los bloqueos se mantengan hasta el **commit**: los desbloques pueden ocurrir antes (como aquí $UX_1(X)$ en el paso 7). La variante que sí los retiene hasta el final es **2PL riguroso**, que además evita el rollback en cascada.

4. Ejercicio 3: Protocolo de marcas temporales

Enunciado. Dadas

- T_1 : $\text{leer}(X)$; $\text{escribir}(X)$; $\text{leer}(Y)$; $\text{escribir}(Y)$;
- T_2 : $\text{leer}(X)$; $\text{escribir}(X)$;

- T_3 : leer(Y); escribir(Y);

dar una planificación con el **protocolo de ordenación por marcas temporales**, mostrando cómo evolucionan $MT-E$ y $MT-L$ sobre X y Y .

4.1. Marco teórico aplicado

A cada transacción T_i se le asigna una marca $MT(T_i)$ que la ordena en el sistema (más chica = más vieja). Cada ítem Q guarda:

- $MT-E(Q)$: mayor marca de las transacciones que escribieron con éxito sobre Q .
- $MT-L(Q)$: mayor marca de las transacciones que leyeron con éxito sobre Q .

Reglas del protocolo:

Lectura: T_i ejecuta $r(Q)$.

- Si $MT(T_i) < MT-E(Q)$: la lectura se rechaza (querría leer una versión que ya fue sobrecrita) y T_i se retrocede.
- En otro caso: se ejecuta y se actualiza $MT-L(Q) \leftarrow \max(MT-L(Q), MT(T_i))$.

Escritura: T_i ejecuta $w(Q)$.

- Si $MT(T_i) < MT-L(Q)$: el valor que produciría T_i ya fue necesario por una lectura posterior. Se rechaza, T_i se retrocede.
- Si $MT(T_i) < MT-E(Q)$: T_i querría escribir una versión obsoleta. Se rechaza, T_i se retrocede.
- En otro caso: se ejecuta y se actualiza $MT-E(Q) \leftarrow MT(T_i)$.

4.2. Asignación inicial

Asigno marcas crecientes a las transacciones según el orden propuesto:

$$MT(T_1) = 1, \quad MT(T_2) = 2, \quad MT(T_3) = 3.$$

Estado inicial de los ítems: $MT-E(X) = MT-L(X) = 0$, $MT-E(Y) = MT-L(Y) = 0$.

4.3. Schedule propuesto y evolución

$$r_1(X), w_1(X), r_2(X), w_2(X), r_1(Y), w_1(Y), r_3(Y), w_3(Y)$$

Paso	Operación	Validación	$MT-E(X)$	$MT-L(X)$
0	inicio	—	0	0
1	$r_1(X)$	$1 \geq MT-E(X)=0 \Rightarrow \text{OK}; MT-L(X)=\max(0,1)=1$	0	0
2	$w_1(X)$	$1 < MT-L(X)=1? \text{ no}; 1 < MT-E(X)=0? \text{ no} \Rightarrow \text{OK}; MT-E(X)=1$	1	1
3	$r_2(X)$	$2 \geq MT-E(X)=1 \Rightarrow \text{OK}; MT-L(X)=\max(1,2)=2$	1	1
4	$w_2(X)$	$2 < MT-L(X)=2? \text{ no}; 2 < MT-E(X)=1? \text{ no} \Rightarrow \text{OK}; MT-E(X)=2$	2	2
5	$r_1(Y)$	$1 \geq MT-E(Y)=0 \Rightarrow \text{OK}; MT-L(Y)=\max(0,1)=1$	2	2
6	$w_1(Y)$	$1 < MT-L(Y)=1? \text{ no}; 1 < MT-E(Y)=0? \text{ no} \Rightarrow \text{OK}; MT-E(Y)=1$	2	2
7	$r_3(Y)$	$3 \geq MT-E(Y)=1 \Rightarrow \text{OK}; MT-L(Y)=\max(1,3)=3$	2	2
8	$w_3(Y)$	$3 < MT-L(Y)=3? \text{ no}; 3 < MT-E(Y)=1? \text{ no} \Rightarrow \text{OK}; MT-E(Y)=3$	2	2

4.4. Resultado final

Sin rollbacks. Estado final:

- $X: MT-E(X) = 2, MT-L(X) = 2.$
- $Y: MT-E(Y) = 3, MT-L(Y) = 3.$

Por qué pasa el test. El orden efectivo de conflictos respeta el orden temporal de las marcas: T_1 aparece antes que T_2 en los conflictos sobre X , y T_1 antes que T_3 en los conflictos sobre Y . Como no hay accesos cruzados (ninguna T_i con $MT(T_i)$ chica intenta operar sobre un ítem “ya tomado” por una con MT mayor), no se viola ninguna de las condiciones de retroceso.

5. Ejercicio 4: UPDATE y ROLLBACK en MySQL

Enunciado. Abrir una conexión al cliente MySQL contra la base del Práctico 1.a, aumentar 20 % el precio de un producto que nadie más esté actualizando, verificar el cambio, hacer **ROLLBACK** y verificar si el cambio quedó almacenado o no.

5.1. Marco teórico aplicado

El experimento ilustra la propiedad de **atomicidad** (la “A” de ACID): una transacción o se aplica completa o no se aplica en absoluto. **ROLLBACK** es la primitiva que materializa esa propiedad cuando la transacción todavía no fue confirmada.

Mientras una transacción esté abierta (entre **START TRANSACTION** y el **COMMIT/ROLLBACK**), las escrituras son visibles *dentro* de ella, pero no son persistentes ni visibles para otras transacciones (en niveles **READ COMMITTED** o más fuertes).

5.2. Ejecución sugerida

Conexión por línea de comandos:

```
mysql -u root -p
```

Una vez dentro del cliente:

```
USE practico1a_mysql;

SET autocommit = 0;
START TRANSACTION;

-- Producto elegido: el 104 esta libre en la carga del practico 1
SELECT cod_producto, descripcion, precio
FROM producto
WHERE cod_producto = 104
FOR UPDATE;           -- toma lock exclusivo de fila

-- Guardar el precio original para comparar
SELECT @precio_original := precio
FROM producto
WHERE cod_producto = 104;

-- Aumentar 20%
UPDATE producto
SET precio = ROUND(precio * 1.20, 2)
WHERE cod_producto = 104;

-- Verificar dentro de la transaccion: debe verse el nuevo precio
```



```
SELECT cod_producto,
       @precio_original AS precio_antes,
       precio           AS precio_despues_update
FROM producto
WHERE cod_producto = 104;

-- Deshacer
ROLLBACK;

-- Verificar despues del rollback: debe volver al precio original
SELECT cod_producto, precio AS precio_despues_rollback
FROM producto
WHERE cod_producto = 104;
```

5.3. Resultado esperado

Con los datos cargados en el Práctico 1, para `cod_producto = 104`:

- Precio inicial: 450000.00
- Tras el UPDATE (visible *sólo dentro* de la transacción): 540000.00
- Tras el ROLLBACK: 450000.00 (vuelve al valor original).

Conclusión: el cambio no quedó persistido. ROLLBACK deshace todas las escrituras realizadas desde START TRANSACTION. Es la atomicidad en acción.

5.4. Verificación opcional desde una segunda sesión

Abriendo otra conexión y consultando `cod_producto = 104` *antes* del ROLLBACK en la primera sesión, en READ COMMITTED o REPEATABLE READ la segunda sesión sigue viendo 450000.00: nunca vio el valor sucio.

6. Ejercicio 5: Dos sesiones MySQL en SERIALIZABLE

Enunciado. Abrir dos conexiones a la base del Práctico 1.a en nivel SERIALIZABLE, intentar modificar el mismo registro de `cliente` desde las dos transacciones y observar qué pasa. Cerrar ambas con COMMIT.

6.1. Marco teórico aplicado

El nivel SERIALIZABLE es el más estricto definido por SQL 92: no permite lecturas sucias, lecturas no repetibles, ni *phantoms*. En MySQL/InnoDB se implementa con bloqueos: lectura compartida (LOCK IN SHARE MODE) implícita en cada SELECT, y bloqueo exclusivo en cada modificación.

Cuando dos transacciones intentan modificar el mismo registro, la segunda queda **esperando** el lock que tiene la primera; la concurrencia sobre esa fila se convierte de hecho en ejecución serial.

6.2. Prueba propuesta (cliente con `nro_cliente = 1`)

Sesión A (Terminal 1)

```
USE practico1a_mysql;

SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SET autocommit = 0;
START TRANSACTION;
```

```
SELECT nro_cliente, direccion
FROM cliente
WHERE nro_cliente = 1;

UPDATE cliente
SET direccion = 'San Martin 123 - A'
WHERE nro_cliente = 1;
```

En este punto la fila `nro_cliente = 1` queda con bloqueo exclusivo de fila tomado por A.

Sesión B (Terminal 2)

```
USE practico1a_mysql;

SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SET autocommit = 0;
START TRANSACTION;

SELECT nro_cliente, direccion
FROM cliente
WHERE nro_cliente = 1;  -- queda esperando (lock S vs X)

UPDATE cliente
SET direccion = 'San Martin 123 - B'
WHERE nro_cliente = 1;  -- queda esperando si paso el SELECT
```

Qué se observa. La sentencia de B queda colgada esperando que A libere el lock. No hay ejecución concurrente real sobre esa fila: el motor las serializa.

Cierre pedido por el enunciado

1. En Sesión A: COMMIT;
2. En Sesión B se desbloquea y la operación pendiente prosigue.
3. En Sesión B: COMMIT;

Resultado final. Verificando con

```
SELECT nro_cliente, direccion FROM cliente WHERE nro_cliente = 1;
```

queda 'San Martin 123 - B' porque B se confirmó después de A. Si la espera supera `innodb_lock_wait_timeout` (50 s por defecto), MySQL aborta la transacción con el error 1205 (Lock wait timeout exceeded).

7. Ejercicio 6: Lectura no repetible

Enunciado. Diseñar una prueba en MySQL donde una misma transacción, leyendo dos veces el mismo registro (sin modificarlo), obtenga valores distintos. Indicar qué niveles de aislamiento lo permiten.

7.1. Marco teórico aplicado

La **lectura no repetible** (*non-repeatable read*) es uno de los tres fenómenos clásicos asociados a niveles de aislamiento débiles. Sucede cuando una transacción T_1 lee un registro, una segunda transacción T_2 lo modifica y commita, y T_1 lee de nuevo el registro encontrando un valor distinto.

Lo que se rompió no es la durabilidad de T_2 —es legítimo que confirme— sino la sensación de “mundo estable” que debería tener T_1 durante su vida.

Nivel	Lectura sucia	No repetible	Phantom
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	No posible	Posible	Posible
REPEATABLE READ	No posible	No posible	Posible (raro)
SERIALIZABLE	No posible	No posible	No posible

7.2. Prueba (producto.cod_producto = 102)

Sesión A (la que sólo lee)

```
USE practico1a_mysql;

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET autocommit = 0;
START TRANSACTION;

SELECT cod_producto, precio
FROM producto
WHERE cod_producto = 102;      -- supongamos 8000.00
```

Sesión B (la que modifica entre lecturas de A)

```
USE practico1a_mysql;
SET autocommit = 0;
START TRANSACTION;

UPDATE producto
SET precio = precio + 500
WHERE cod_producto = 102;

COMMIT;      -- confirma la modificacion
```

Vuelta a Sesión A: segunda lectura del mismo registro

```
SELECT cod_producto, precio
FROM producto
WHERE cod_producto = 102;      -- ahora 8500.00

COMMIT;
```

A ve un valor distinto en dos lecturas *dentro de la misma transacción* sin haberlo modificado: lectura no repetible.

7.3. Niveles de aislamiento que lo permiten

En MySQL/InnoDB:

- READ UNCOMMITTED y READ COMMITTED: lo permiten.
- REPEATABLE READ y SERIALIZABLE: no lo permiten. En REPEATABLE READ (el default de InnoDB) las lecturas consistentes se sirven contra un *snapshot* fijo en el primer SELECT, así que la segunda lectura devuelve el mismo valor que la primera incluso si B confirmó cambios en el medio.

Verificación complementaria. Repitiendo la prueba con REPEATABLE READ en la sesión A:

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

la segunda lectura de A devuelve el valor original (8000.00), demostrando que el snapshot oculta los commits posteriores de B.

8. Ejercicio 7: Deferrable Constraints en PostgreSQL

Enunciado. Sobre el esquema

- cliente(dni, nombre, apellido, dirección, tarifa)
- automovil(patente, marca, modelo, dni, nro_categoria)
- categoría(nro_categoria, tasa)
- taller(nro_taller, nombre, dirección)
- accidente(nro_accidente, dni, patente, código_taller, fecha, costo)

considerar una transacción que inserta primero en accidente (cuyo patente todavía no existe) y después inserta en automovil la patente faltante. Resolver:

- (a) ¿Qué ocurre con el esquema normal?
- (b) Modificar el esquema para que la transacción no falle.
- (c) Modificar el esquema para que sí falle.

8.1. Marco teórico aplicado

Las **Foreign Keys** en SQL pueden chequearse de dos maneras:

- **Inmediata** (default): el motor verifica la integridad después de cada sentencia. Si la constraint no se cumple en ese momento, la sentencia falla.
- **Diferida** (DEFERRABLE INITIALLY DEFERRED): la verificación se posterga hasta el COMMIT. Permite operaciones transitoriamente inconsistentes dentro de la transacción, siempre que al confirmar todo cuadre.

La transacción del enunciado tiene una inconsistencia transitoria (insertar un accidente que referencia un automóvil aún no cargado), por lo que sólo funciona si la FK accidente.patente → automovil.patente es deferible.

8.2. a) Comportamiento con esquema normal

Con FK inmediata, el primer INSERT falla porque accidente.patente = 'PAT1' no existe en automovil:

```
BEGIN;

INSERT INTO practico1c.accidente
(nro_accidente, dni, patente, nro_taller, fecha, costo)
VALUES
(1, 30111222, 'PAT1', 1, DATE '2009-10-10', 234);
-- ERROR: insert or update on table "accidente" violates foreign key
-- constraint "fk_accidente_automovil"

INSERT INTO practico1c.automovil
(patente, marca, modelo, dni, nro_categoria)
VALUES ('PAT1', 'FIAT', 2010, 30111222, 1);
```

```
COMMIT;           -- ya no aplica: la transaccion esta abortada
```

8.3. b) Esquema que evita la excepción

Hacer la FK deferible y diferirla por defecto:

```
ALTER TABLE practico1c.accidente
  ALTER CONSTRAINT fk_accidente_automovil
  DEFERRABLE INITIALLY DEFERRED;
```

Con eso la transacción anterior funciona tal cual:

```
BEGIN;

-- (Opcional, ya esta en DEFERRED por la ALTER)
SET CONSTRAINTS fk_accidente_automovil DEFERRED;

INSERT INTO practico1c.accidente
  (nro_accidente, dni, patente, nro_taller, fecha, costo)
VALUES (1, 30111222, 'PAT1', 1, DATE '2009-10-10', 234);

INSERT INTO practico1c.automovil
  (patente, marca, modelo, dni, nro_categoria)
VALUES ('PAT1', 'FIAT', 2010, 30111222, 1);

COMMIT;           -- recién aquí se valida la FK; en este momento PAT1 existe
```

No hay excepción.

8.4. c) Esquema que fuerza excepción

Volver la FK a no deferible (chequeo inmediato):

```
ALTER TABLE practico1c.accidente
  ALTER CONSTRAINT fk_accidente_automovil
  NOT DEFERRABLE;
```

La misma transacción vuelve a fallar en el primer INSERT exactamente como en (a).

8.5. Consulta de control

Para inspeccionar el estado de las constraints:

```
SELECT conname, condeferrable, condeferred
FROM pg_constraint
WHERE conrelid = 'practico1c.accidente'::regclass
  AND conname = 'fk_accidente_automovil';
```

- condeferrable = true, condeferred = true $\Rightarrow$ DEFERRABLE INITIALLY DEFERRED (caso b).
- condeferrable = false $\Rightarrow$ NOT DEFERRABLE (caso c).

9. Ejercicio 8: MVCC en PostgreSQL (`xmin` / `xmax`)

Enunciado. Sobre la base del Práctico SQL (ejercicio b), diseñar y ejecutar una prueba donde dos transacciones vean valores distintos para la misma fila, observando los valores `xmin` y `xmax`.

9.1. Marco teórico aplicado

PostgreSQL implementa **MVCC** (*Multiversion Concurrency Control*): cada `UPDATE` no machaca la versión vieja sino que crea una versión nueva. Cada tupla tiene dos columnas de sistema:

- `xmin`: identificador (XID) de la transacción que *insertó* esa versión.
- `xmax`: XID de la transacción que la *invalida* (vía `UPDATE` o `DELETE`); 0 si todavía está viva.

Las lecturas no toman bloqueo: el snapshot de la transacción lectora decide cuál de las versiones ver. Esto permite que dos transacciones concurrentes vean valores distintos para la misma fila sin esperar la una a la otra.

9.2. Prueba (`vehiculo.patente = 'AA123BB'`)

Sesión A — transacción T_A

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT txid_current() AS tx_a;

SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';

UPDATE vehiculo
SET saldo_actual = saldo_actual + 100
WHERE patente = 'AA123BB';

SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';

-- NO HACER COMMIT TODAVIA
```

Lo esperado dentro de T_A :

- Antes del `UPDATE`: se ve el valor original. Esa versión tiene `xmin` con el XID de la transacción que la creó y `xmax` = 0.
- Después del `UPDATE`: la versión visible para T_A es la nueva, con `xmin` = `tx_a` y `xmax` = 0. La versión vieja queda con `xmax` = `tx_a`, pero T_A ya no la ve.

Sesión B — transacción T_B , en paralelo, antes del commit de A

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT txid_current() AS tx_b;

SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';
```

Lo esperado dentro de T_B :

- T_B ve el *valor anterior* (el que estaba antes del UPDATE de A), porque el snapshot de B no incluye los cambios no confirmados de A.
- En esa versión, `xmin` corresponde a la transacción que creó la tupla original; `xmax` puede mostrar `tx_a` (la transacción que está invalidando esa versión), pero para B sigue siendo la versión visible porque T_A todavía no confirmó.

Acá está la observación pedida por el enunciado: T_A y T_B ven valores distintos para la misma fila, simultáneamente y sin que ninguna espere.

Cierre y segunda lectura

En Sesión A:

```
COMMIT;
```

En Sesión B (que sigue en su transacción READ COMMITTED):

```
SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';

COMMIT;
```

En READ COMMITTED, esta segunda lectura de B toma un snapshot nuevo y ya ve el valor confirmado por A.

Si en lugar de READ COMMITTED la sesión B usara REPEATABLE READ o SERIALIZABLE, su snapshot quedaría fijado al primer SELECT: la segunda lectura mostraría todavía el valor anterior. Si B intentara modificar la misma fila vería el error “*could not serialize access due to concurrent update*”.

9.3. Interpretación de xmin/xmax

- Una versión *viva* tiene `xmax` = 0.
- Un UPDATE produce dos versiones:
 - versión vieja: `xmax` = XID del UPDATE; deja de ser visible para snapshots que comienzan después del commit del update.
 - versión nueva: `xmin` = XID del UPDATE, `xmax` = 0.
- Un DELETE marca la fila con `xmax` = XID del DELETE sin crear nueva versión.
- Las versiones muertas se eliminan más tarde con VACUUM (o el autovacuum).

9.4. Restaurar el dato (opcional)

```
UPDATE vehiculo
SET saldo_actual = saldo_actual - 100
WHERE patente = 'AA123BB';
```

10. Conclusiones

El práctico recorre los dos planos en los que se piensa la concurrencia de transacciones:

1. **Plano teórico** (Ejercicios 1–3): cómo se verifica que un schedule es “equivalente a una ejecución serial” aunque haya intercalado real. La herramienta es el grafo de precedencia (test de serializabilidad por conflicto), y los dos protocolos que garantizan schedules serializables son el bloqueo de dos fases y la ordenación por marcas temporales.
2. **Plano práctico** (Ejercicios 4–8): cómo se materializan estos conceptos en motores reales. ROLLBACK aplica atomicidad. Los *niveles de aislamiento* permiten relajar la serializabilidad estricta para ganar concurrencia, asumiendo determinados fenómenos (lectura sucia, no repetible, phantom). Las *constraints diferidas* permiten transacciones con inconsistencias intermedias que cierran bien al commit. Y el MVCC de PostgreSQL muestra cómo distintos snapshots permiten a varias transacciones ver versiones distintas de la misma fila sin bloquearse entre sí.

La idea común detrás del práctico es: *las transacciones aíslan al programador del caos de la concurrencia, pero ese aislamiento tiene costos*. Los protocolos y los niveles de aislamiento son los manijos que el sistema ofrece para regular ese trade-off entre consistencia y rendimiento.